Classes, Structs, and Access Specifiers

Theory, Edge Cases & Code Examples

- ✓ Identical Except Default Access
- ✓ Compile-Time Visibility Control
- ✓ Why Access Control Matters
- ✓ Inheritance and Access Specifiers
- ✓ Memory Layout and Access Specifiers

→ Swipe to learn →

Classes, Structs, and Access Specifiers

Theory

📌 1. Classes vs Structs - Identical Except Default Access (1/2)

Definition: Classes and structs are user-defined types that bundle data (members) and functions (methods) together. They are **functionally identical** - the ONLY differences are their default access specifiers and default inheritance modes.

Core Differences Table:

Aspect	`struct`	`class`
Default member access	`public`	`private`
Default inheritance mode	`public`	`private`
Typical usage	Simple data containers (PODs)	Encapsulated objects with behavior
Can have constructors?	✅ Yes	✅ Yes
Can have virtual functions?	✅ Yes	✅ Yes
Can be used as base class?	✅ Yes	✅ Yes
Memory layout difference	**None** - identical layout	**None** - identical layout

Classes, Structs, and Access Specifiers

Theory

📌 1. Classes vs Structs - Identical Except Default Access (2/2)

Before/After Examples:

```
// ✅ Struct - public by default
struct Point {
    int x, y;  // Implicitly public
    void print() { std::cout << x << ", " << y; }
};

Point p;
p.x = 10;  // ✅ Direct access allowed

// ✅ Class - private by default
class Point2 {
    int x, y;  // Implicitly private
public:
    void setX(int val) { x = val; }
    void print() { std::cout << x << ", " << y; }
};

Point2 p2;
// p2.x = 10;  // ❌ Error: x is private
p2.setX(10);   // ✅ Use public setter
```

When to Use Which:

Use `struct` when	Use `class` when
Simple data container (POD)	Object requires encapsulation
All members should be public	Need to enforce invariants
C compatibility needed	Implementing design patterns
No behavior, just data	Complex behavior and state
Aggregate initialization desired	Constructor validation needed

Classes, Structs, and Access Specifiers

Theory

📌 2. Access Specifiers - Compile-Time Visibility Control (1/2)

Definition: Access specifiers are keywords that control the visibility and accessibility of class members, enforcing **encapsulation** at compile time.

Three Access Levels:

Specifier	Accessible From	Use Case	Memory Impact
`public`	Anywhere (inside class, derived classes, outside)	Public interface, API	None
`protected`	Inside class + derived classes only	Protected interface for inheritance	None
`private`	Inside class only	Internal implementation details	None

Access Control Matrix:

Context	Can Access Public	Can Access Protected	Can Access Private
Same class	✅	✅	✅
Derived class	✅	✅	❌
Outside code	✅	❌	❌
Friend function/class	✅	✅	✅

Classes, Structs, and Access Specifiers

Theory

📌 2. Access Specifiers - Compile-Time Visibility Control (2/2)

Practical Example:

```
class BankAccount {
private:
    double balance;        // Only accessible within BankAccount
    void validateAmount(double amt) { /* ... */ }

protected:
    std::string accountType;  // Accessible in derived classes

public:
    void deposit(double amt) {  // Public API
        if (amt > 0) balance += amt;
    }
    double getBalance() const { return balance; }
};

class SavingsAccount : public BankAccount {
    void test() {
        // balance = 100;        // ❌ Error: private in base
        accountType = "Savings"; // ✅ OK: protected accessible
        deposit(50);             // ✅ OK: public accessible
    }
};
```

Critical Characteristics:

- **Compile-time only:** Access control is checked during compilation, not at runtime
- **No memory impact:** Access specifiers don't affect object size or memory layout
- **Per-class, not per-object:** Member functions can access private members of ANY instance of the same class
- **Not security:** Can be bypassed with unsafe pointer casts (undefined behavior)

Classes, Structs, and Access Specifiers

Theory

📌 3. Encapsulation - Why Access Control Matters (1/2)

Definition: Encapsulation is the OOP principle of bundling data and methods together while hiding internal implementation details from external code.

Benefits of Access Control:

Benefit	Description	Example
Invariant enforcement	Prevent invalid state	`balance` can't go negative if only modified through `deposit()/withdraw()`
Interface stability	Change internals without breaking users	Can change `balance` storage from `double` to `Cents` class
Reduced coupling	Users depend on interface, not implementation	External code doesn't know or care about internal data structures
Compile-time safety	Catch misuse at compile time	Attempt to modify `private` member causes compiler error

Classes, Structs, and Access Specifiers

Theory

📌 3. Encapsulation - Why Access Control Matters (2/2)

Encapsulation Patterns:

```
// ✅ Good encapsulation - private data, public interface
class GoodDesign {
private:
    int value;
    bool isValid() const { return value >= 0; }

public:
    void setValue(int v) {
        if (v < 0) throw std::invalid_argument("Negative value");
        value = v;
    }
    int getValue() const { return value; }
};

// ❌ Poor encapsulation - public data
class PoorDesign {
public:
    int value;  // Anyone can modify without validation
};
```

Interview Relevance:

- **Common question:** "What's the difference between struct and class?" → Default access
- **Design questions:** "How would you design a class to ensure X invariant?" → Use private + validation
- **Inheritance questions:** How access specifiers interact with inheritance modes
- **Friend declarations:** When and why to break encapsulation intentionally

Classes, Structs, and Access Specifiers

Edge Cases

📌 1. Inheritance and Access Specifiers

Both `struct` and `class` can be used as base classes, and the type doesn't affect inheritance semantics—only the default inheritance specifier changes. When you inherit from a `struct`, the default inheritance is public. When you inherit from a `class`, the default is private. This can lead to surprising behavior if you forget to specify the inheritance type explicitly.

```
struct Base {
    void foo() {}
};

// Inherits publicly by default (struct)
struct Derived1 : Base {
    void bar() { foo(); }  // ✅ Accessible
};

class Base2 {
public:
    void foo() {}
};

// Inherits privately by default (class)
class Derived2 : Base2 {
    void bar() { foo(); }  // ✅ Accessible within class
};

// But from outside:
// Derived2 d;
// d.foo();  // ❌ Error: foo is inaccessible (private inheritance)
```

This demonstrates that private inheritance makes all base class members private in the derived class, regardless of their original access level.

Classes, Structs, and Access Specifiers

Edge Cases

📌 2. Memory Layout and Access Specifiers

Access specifiers are purely a compile-time enforcement mechanism and have **no impact on memory layout or object size**. Members are laid out in memory in the order they're declared, regardless of whether they're public, protected, or private. However, compilers may add padding between members for alignment purposes, and access specifier boundaries don't prevent this optimization.

```
struct S1 {
    int a;      // offset 0
    int b;      // offset 4
};

class S2 {
private:
    int a;      // offset 0
public:
    int b;      // offset 4
};

// sizeof(S1) == sizeof(S2), same layout
```

This means you cannot use access specifiers to control memory layout—you need alignment attributes or packing directives for that.

Classes, Structs, and Access Specifiers

Edge Cases

📌 3. Private Members and Pointer Hacks

While access specifiers provide compile-time protection, they don't provide runtime security. It's technically possible to access private members through unsafe pointer arithmetic or casting, though this is undefined behavior and violates C++'s type system.

```
class Secret {
private:
    int value = 42;
};

int main() {
    Secret s;
    int* ptr = reinterpret_cast<int*>(&s);  // ❌ Dangerous hack
    std::cout << *ptr;  // May print 42, but undefined behavior
}
```

This "works" because the object has a contiguous memory layout, and `value` is at a predictable offset. However, this approach is implementation-dependent, breaks encapsulation, and should never be used in production code. It's only relevant in reverse engineering, security exploits, or debugging scenarios.

Classes, Structs, and Access Specifiers

Edge Cases

📌 4. Virtual Functions and Access Control

Access specifiers control who can **call** a function, but they don't affect whether the function participates in virtual dispatch. Even private virtual functions are placed in the vtable and can be overridden by derived classes (if they have access).

```
class Base {
private:
    virtual void secret() { std::cout << "Base::secret\n"; }
public:
    void callSecret() { secret(); }  // Calls through vtable
};

class Derived : public Base {
    // Cannot override secret() because it's private in Base
    // But if we could, it would use virtual dispatch
};
```

The vtable is a runtime mechanism for polymorphism, while access control is compile-time, so they operate independently.

Classes, Structs, and Access Specifiers

Edge Cases

📌 5. Friend Declarations Override Access Control

The `friend` keyword allows external functions or classes to access private and protected members, effectively bypassing access control. This is useful for tightly coupled classes but should be used sparingly as it breaks encapsulation.

```
class Box {
private:
    int contents = 100;
    friend class Inspector;
    friend void reveal(const Box&);
};

class Inspector {
public:
    void inspect(const Box& b) {
        std::cout << b.contents;  // ✅ Allowed
    }
};

void reveal(const Box& b) {
    std::cout << b.contents;  // ✅ Allowed
}
```

Friendship is not inherited and must be explicitly granted, making it a powerful but potentially dangerous feature.

Classes, Structs, and Access Specifiers

Code Examples

📌 1. Basic Struct vs Class Usage

This demonstrates the fundamental difference: struct members are public by default, allowing direct access, while class members are private and require accessor methods for encapsulation.

```
#include <iostream>
using namespace std;

struct MyStruct {
    int x;  // ✅ Public by default
    void show() { cout << "x = " << x << endl; }
};

class MyClass {
    int y;  // ❌ Private by default
public:
    void setY(int val) { y = val; }
    void show() { cout << "y = " << y << endl; }
};

int main() {
    MyStruct s;
    s.x = 10;  // ✅ Direct access allowed
    s.show();

    MyClass c;
    // c.y = 5;  // ❌ Error: 'y' is private
    c.setY(20);  // ✅ Use public setter
    c.show();
}
```

Classes, Structs, and Access Specifiers

Code Examples

📌 2. Inheritance with Different Access Specifiers

The inheritance specifier determines the maximum access level for inherited members—public inheritance preserves access levels, protected makes everything at most protected, and private makes everything private in the derived class.

```
class Base {
protected:
    int prot_member = 10;
private:
    int priv_member = 20;
public:
    int pub_member = 30;
};

class PublicDerived : public Base {
    void test() {
        prot_member = 1;  // ✅ Protected remains protected
        pub_member = 2;   // ✅ Public remains public
        // priv_member = 3;  // ❌ Private never accessible
    }
};

class ProtectedDerived : protected Base {
    void test() {
        prot_member = 1;  // ✅ Still accessible
        pub_member = 2;   // ✅ Becomes protected in this class
    }
};

class PrivateDerived : private Base {
    void test() {
        prot_member = 1;  // ✅ Accessible but becomes private
        pub_member = 2;   // ✅ Accessible but becomes private
    }
};
```

Classes, Structs, and Access Specifiers

Code Examples

📌 3. Struct as POD (Plain Old Data)

Structs are ideal for simple data containers that need C compatibility, trivial copying, or memory-mapped I/O. They don't enforce encapsulation but provide convenience.

```
struct Point {
    double x;
    double y;
    double z;
};

struct Color {
    unsigned char r, g, b, a;
};

// POD structs can be initialized with aggregate initialization
Point p1 = {1.0, 2.0, 3.0};
Color c1 = {255, 0, 0, 255};

// Useful for C interop and memory mapping
void processPoint(Point* p) {
    // Can safely cast, copy via memcpy, etc.
}
```

Classes, Structs, and Access Specifiers

Code Examples

📌 4. Class with Proper Encapsulation

This demonstrates proper use of a class: private data members ensure the balance can't be manipulated directly, and public methods provide controlled access with validation.

```
class BankAccount {
private:
    double balance;
    string accountNumber;

    bool validateAmount(double amount) {
        return amount > 0;
    }

public:
    BankAccount(string accNum) : balance(0), accountNumber(accNum) {}

    bool deposit(double amount) {
        if (!validateAmount(amount)) return false;
        balance += amount;
        return true;
    }

    bool withdraw(double amount) {
        if (!validateAmount(amount) || amount > balance) return false;
        balance -= amount;
        return true;
    }

    double getBalance() const { return balance; }
};
```

Classes, Structs, and Access Specifiers

Code Examples

📌 5. Protected Members in Inheritance Hierarchies

Protected members strike a balance between private (too restrictive for inheritance) and public (too permissive). They allow derived classes to access implementation details while hiding them from external code.

```
class Animal {
protected:
    string species;
    int age;

    void incrementAge() { age++; }

public:
    Animal(string s, int a) : species(s), age(a) {}
    virtual void makeSound() = 0;
    int getAge() const { return age; }
};

class Dog : public Animal {
public:
    Dog(int a) : Animal("Canine", a) {}

    void makeSound() override {
        cout << "Woof!" << endl;
    }

    void birthday() {
        incrementAge();  // ✅ Can access protected method
        cout << species << " is now " << age << endl;
    }
};
```

Classes, Structs, and Access Specifiers

Code Examples

📌 6. Friend Function for Operator Overloading

Friend functions are commonly used for operator overloading when the operator needs symmetric access to private data of both operands.

```
class Complex {
private:
    double real, imag;

public:
    Complex(double r, double i) : real(r), imag(i) {}

    // Friend function can access private members
    friend Complex operator+(const Complex& a, const Complex& b);
    friend ostream& operator<<(ostream& os, const Complex& c);
};

Complex operator+(const Complex& a, const Complex& b) {
    return Complex(a.real + b.real, a.imag + b.imag);  // ✅ Access private
}

ostream& operator<<(ostream& os, const Complex& c) {
    os << c.real << " + " << c.imag << "i";  // ✅ Access private
    return os;
}
```

Classes, Structs, and Access Specifiers

Code Examples

📌 7. Access Control with Virtual Functions

This shows that you can only override virtual functions you have access to, but all virtual functions participate in dynamic dispatch regardless of access level.

```
class Base {
public:
    virtual void publicVirtual() {
        cout << "Base::publicVirtual" << endl;
    }

protected:
    virtual void protectedVirtual() {
        cout << "Base::protectedVirtual" << endl;
    }

private:
    virtual void privateVirtual() {
        cout << "Base::privateVirtual" << endl;
    }

public:
    void callAll() {
        publicVirtual();
        protectedVirtual();
        privateVirtual();
    }
};

class Derived : public Base {
public:
    void publicVirtual() override {  // ✅ Can override public
        cout << "Derived::publicVirtual" << endl;
    }

protected:
    void protectedVirtual() override {  // ✅ Can override protected
        cout << "Derived::protectedVirtual" << endl;
    }

    // Cannot override privateVirtual - no access to it
};
```

Classes, Structs, and Access Specifiers

Code Examples

📌 8. Common Mistake - Forgetting Access Specifier in Class

Always be explicit about access specifiers in classes to avoid confusion. Placing private members at the end is a common convention that improves readability.

```
class Employee {
    string name;  // ❌ Private by default - common mistake
    int id;       // ❌ Private by default
public:
    void setName(string n) { name = n; }
};

// Better:
class Employee2 {
public:
    void setName(string n) { name = n; }
private:
    string name;  // ✅ Explicit and clear
    int id;
};
```

What You Learned

Key Takeaways

- ✓ Identical Except Default Access
- ✓ Compile-Time Visibility Control
- ✓ Why Access Control Matters
- ✓ Inheritance and Access Specifiers

→ Swipe to learn →